

# The ClipGrow Model

Every actor, every step, every decision point — from the moment someone is handed a login to the moment their clip becomes frozen financial history.

---

Compiled 28 Aug 2026   Reflects `feature/clipper-platform`

Read top-to-bottom or jump by section

---

## PART 0

# Orientation

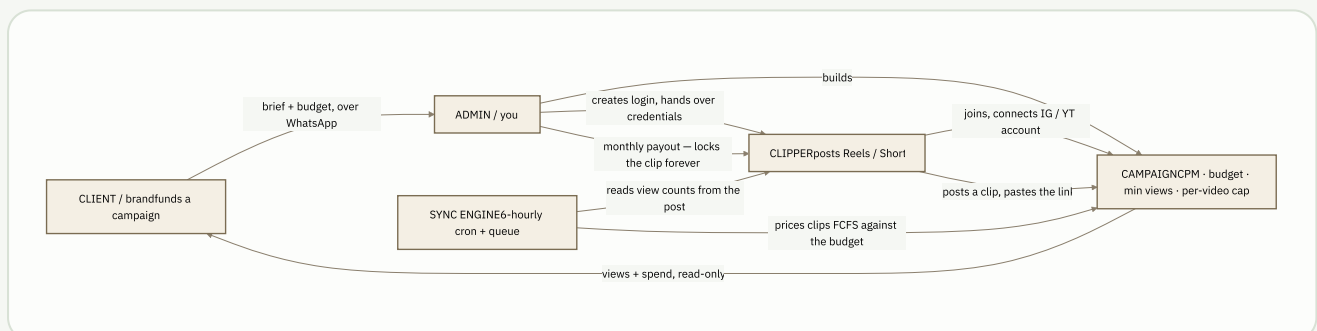
### 0.1 What ClipGrow is

ClipGrow is a **two-sided marketplace with a managed middle**. Brands (*Clients*) fund campaigns. Creators (*Clippers*) cut short vertical videos — Instagram Reels, YouTube Shorts — that promote the brand, and post them from their own accounts. ClipGrow reads the real view count on each post and pays the clipper per thousand verified views. You (the *Admin*) sit in the middle: you sign the brand, build the campaign, recruit and vet clippers, and settle the money once a month.

Technically it is a single **Cloudflare Worker** with a **D1 (SQLite)** database and plain HTML dashboards — no build step, no framework, no ORM. Instagram data comes through Meta's *Instagram API with Instagram Login*; YouTube data through *Google OAuth* + the YouTube Data API.

## 0.2 The four actors

| Actor                    | Enters at                           | Can do                                                                                                                   | Session                                                  |
|--------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Clipper</b>           | <code>/clipper</code><br>(unlisted) | Join campaigns, connect one IG + one YT account per campaign, submit clips, watch earnings climb, see the leaderboard    | 30-day cookie, role <code>clipper</code>                 |
| <b>Client</b><br>(brand) | <code>/client</code><br>(unlisted)  | <b>Read only.</b> See views, budget and spend for the campaigns granted to them — nothing about what any clipper is owed | 30-day cookie, role <code>client</code>                  |
| <b>Admin</b><br>(you)    | <code>/admin</code>                 | Everything: campaigns, approvals, moderation, payouts, sync, diagnostics, exports                                        | 30-day cookie, role <code>admin</code> , single password |
| <b>The engine</b>        | cron & queue                        | Every 6 hours: import new posts, refresh view counts, re-price every open campaign. No human in the loop                 | —                                                        |



The whole model on one page. The brand's money flows in through you and out to clippers; view counts flow the other way and become the price.

## 0.3 The rule that governs everything

### LOAD-BEARING INVARIANT

Once a clip is paid, it is locked at that figure forever. `locked_earning` is what the books say was owed; later view growth never changes it, a CPM change never re-prices it, a budget cut never claws it back. Every write in the codebase that could move an `earning` value carries `WHERE id = ? AND locked_at IS NULL`, and the caller checks how many rows actually changed. A locked clip is financial history — it must never be silently overwritten.

## 0.4 Glossary

| Term                                        | Means                                                                                                                                                                                                                                                 |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Campaign</b>                             | A brand's funded brief. Carries a <b>CPM</b> (rupees per 1,000 views), a total <b>budget</b> , a <b>min views</b> threshold (default 1,000 — a clip below it earns nothing), an optional per-video <b>cap</b> , and an <b>allowed platforms</b> list. |
| <b>Blueprint</b>                            | The campaign's brief content — objective, guidelines, footage links, CTA, tags. Can be parsed straight out of a Word <code>.docx</code> . The per-video cap ( <code>max_payout</code> ) lives here.                                                   |
| <b>Participation</b>                        | One clipper's membership of one campaign. Status: <code>active</code> <code>paused</code> <code>kicked</code> .                                                                                                                                       |
| <b>Submission</b> / clip                    | One posted video tracked against one campaign. Has <code>views</code> , <code>earning</code> , a <code>status</code> , and — once paid — <code>locked_at</code> / <code>locked_earning</code> .                                                       |
| <b>Social account</b>                       | A connected Instagram account or YouTube channel, with its OAuth token. Status: <code>connected</code> <code>needs_reauth</code> <code>revoked</code> .                                                                                               |
| <b>Access request</b><br>("tester request") | The manual approval gate a clipper passes <i>before</i> the Connect button appears. Tracks <code>requested</code> → <code>confirmed</code> / <code>rejected</code> .                                                                                  |
| <b>Refresh job</b>                          | One logical sync of many accounts, run across as many Worker invocations as it takes. Chained through a Cloudflare Queue.                                                                                                                             |

---

### PART 1

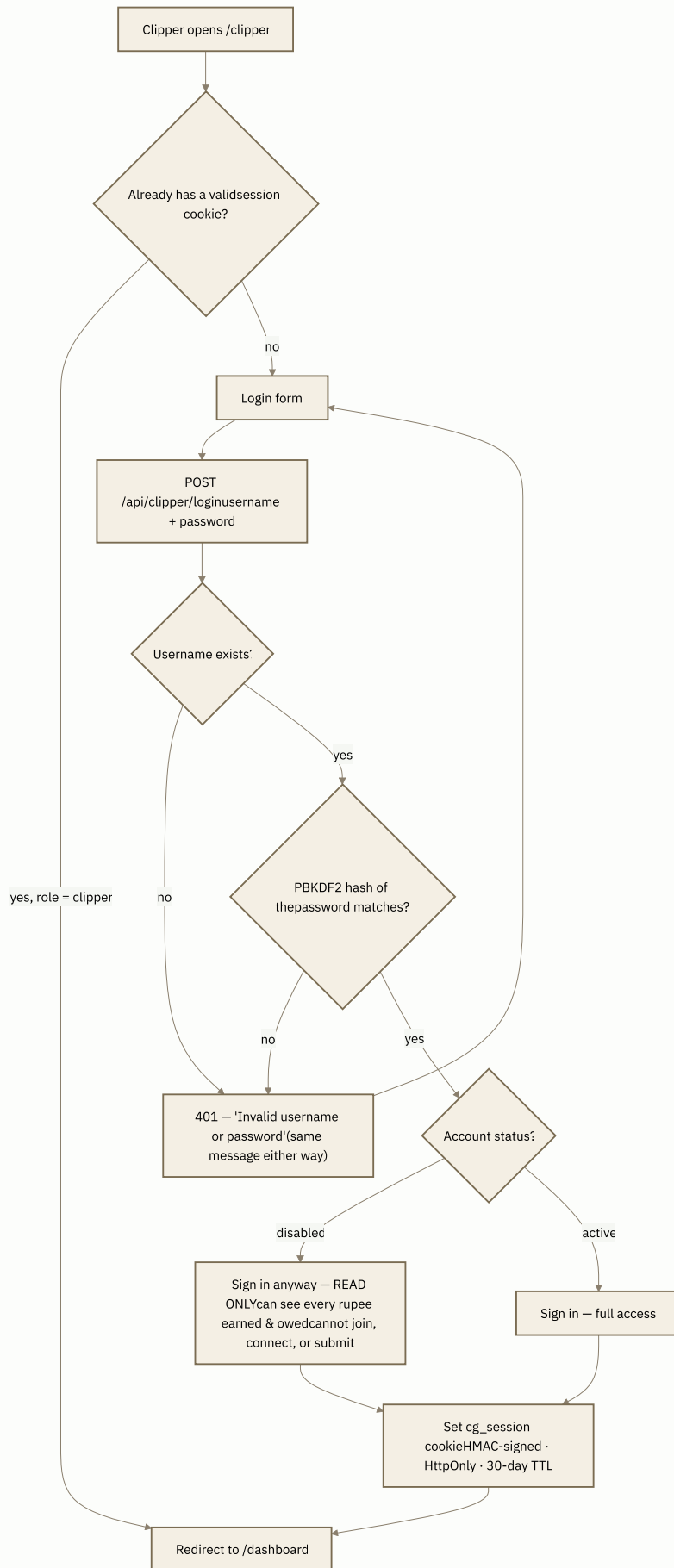
## The Clipper

This is the journey with all the authentication in it, so it gets the most detail. A clipper is recruited into the community, handed a login, joins a campaign, connects the account they will post from, posts clips, and gets paid once a month.

### 1.1 Getting in & logging in

Clippers are recruited through the **WhatsApp community** and the **Discord server** — that is where campaigns, source footage, CPM rates and style guides are posted. There is **no public sign-up**. When someone is ready, **you create their account by hand** in the admin panel ( `POST /api/admin/clippers` — username, a starting password, display name) and hand them the two

credentials directly. The clipper area lives at `/clipper` and nothing on the marketing site links to it.



**Clipper login.** A disabled account is deliberately *not* locked out – it keeps read access so someone can always verify a balance still owed to them.

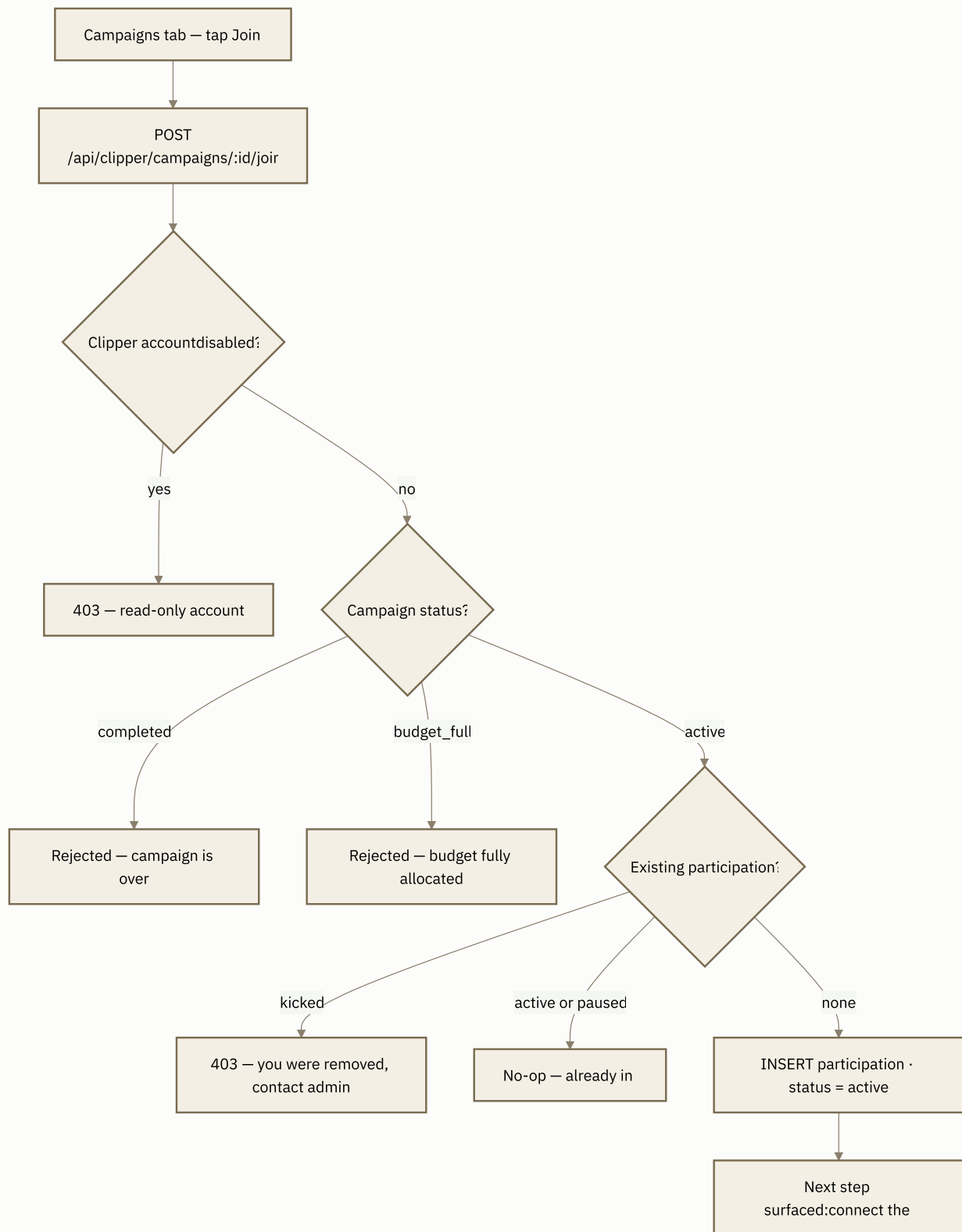
## 1.2 How the login actually works

There are no sessions in the database. The cookie *is* the session:

- **Password storage** — PBKDF2-SHA256, 100,000 iterations, per-user random salt. Verification is constant-time.
- **The cookie** — `cg_session` holds a JSON payload `{role, sub, exp}`, base64url-encoded, with an **HMAC-SHA256 signature** using `SESSION_SECRET`. It is `HttpOnly`, `Secure`, `SameSite=Lax`, 30-day TTL. Tamper with the payload and the signature check fails; the Worker treats it as no session.
- **Gating happens at the edge** — `/dashboard`, `/tracker`, `/client-dashboard` are refused by the Worker without a matching role. The HTML never even reaches an anonymous visitor. An unauthenticated hit is redirected to *the login form for the area they were trying to reach*.
- **Every API route re-checks** the session and the clipper's current status on every request — disabling someone takes effect on their next click, not their next login.

## 1.3 Joining a campaign

The **Campaigns** tab lists every campaign that is not `completed`. Joining is one tap and just records a `participation` row. The gates:



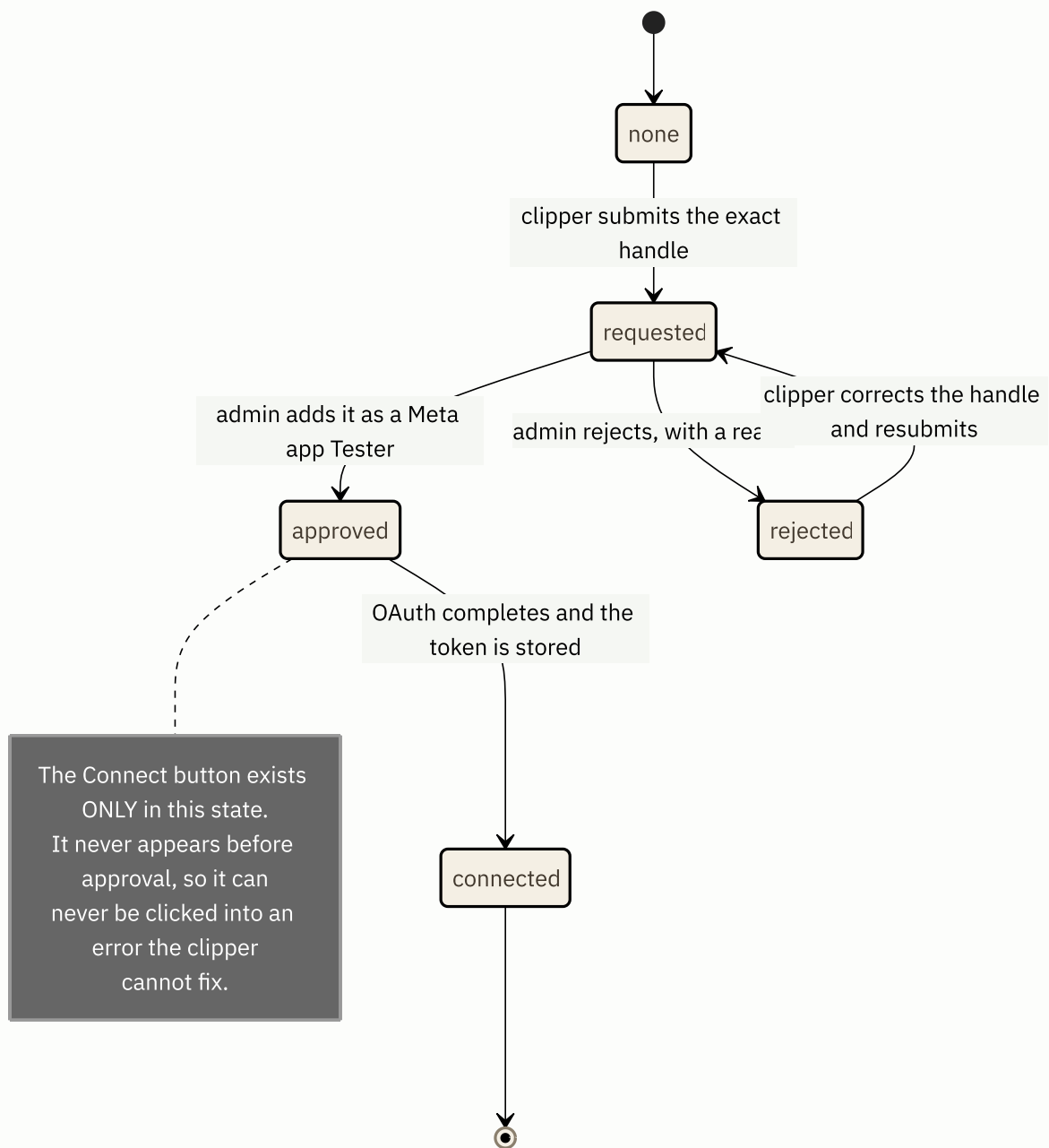
**Join a campaign.** *Joining does nothing on its own — it just unlocks the connect-and-submit flow for that campaign.*

## 1.4 Connecting Instagram

A campaign is tied to **one Instagram account**. Before a clipper can connect, two things must be true, and the UI enforces them as an explicit, visible sequence so the *Connect* button never appears until connecting will actually work:

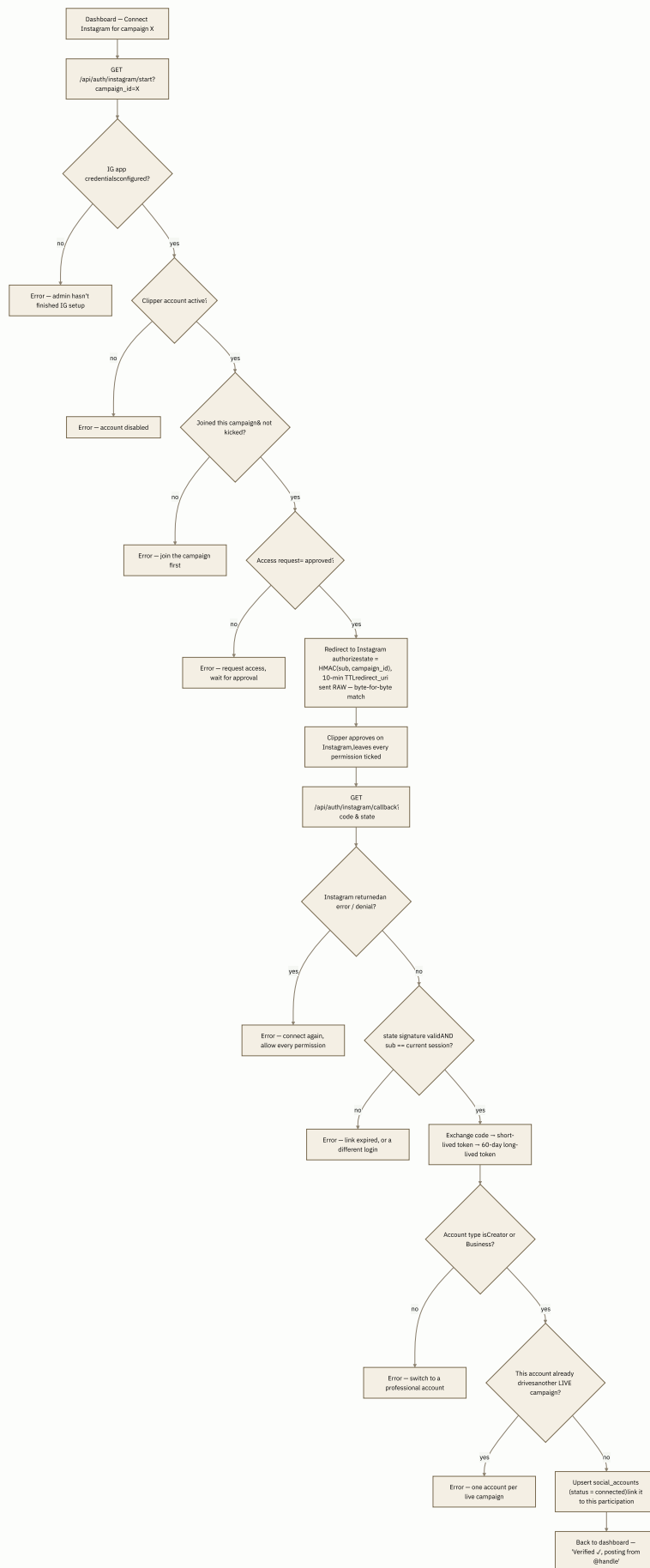
1. The Meta app is still in **Development Mode** (App Review not yet passed), so the Instagram account **must be an accepted app Tester**. You add testers by hand in the Meta dashboard; the clipper accepts the invite in the Instagram app.
2. The account must be a **Professional** account (Creator or Business) — Instagram shares view-count insights for nothing else.





**The access-request state machine (Instagram).** *An approved request cannot be silently re-pointed at a different handle — the account you vetted and the account they connect are the same one.*

Once approved, the clipper hits *Connect Instagram* and the OAuth flow runs. Every branch below is enforced **server-side**, not just by hiding a button — the start URL is a plain link a clipper could keep from an old session.



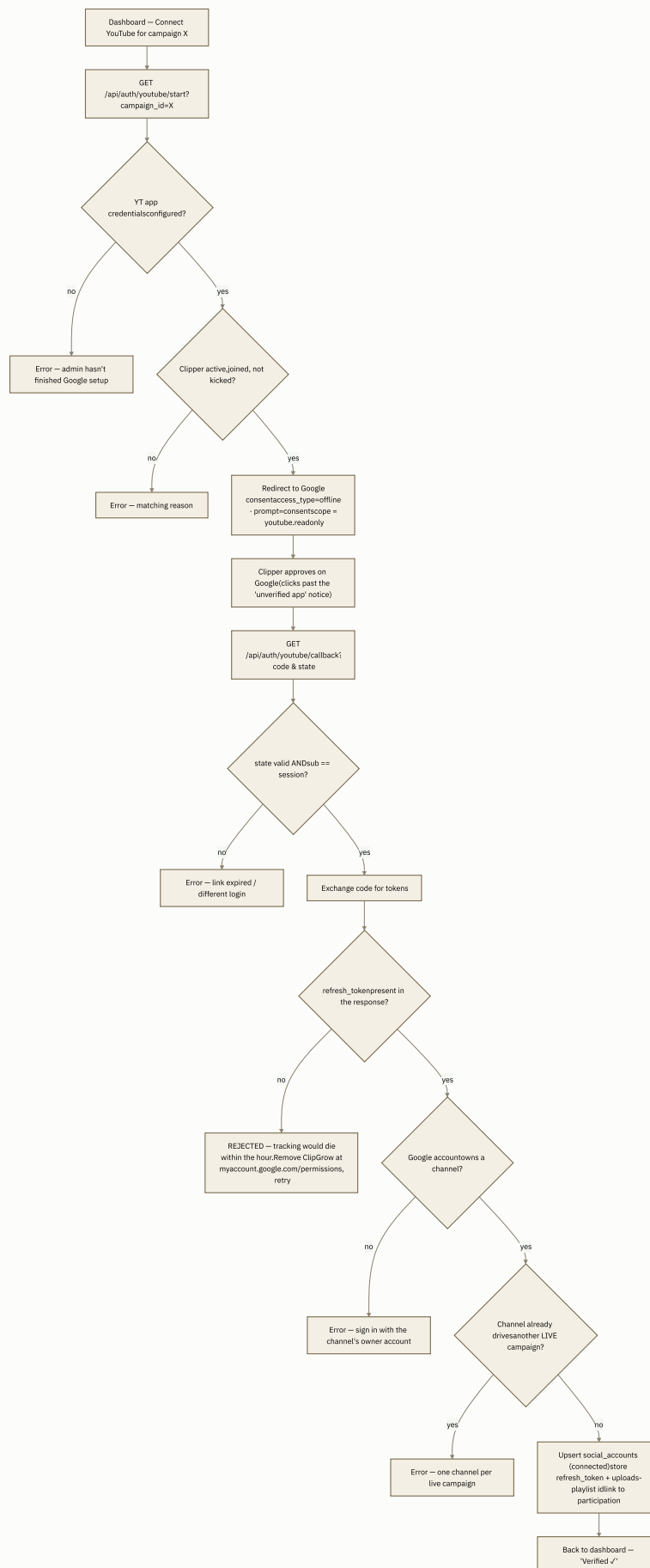
Connect Instagram, full decision tree. The `redirect_uri` is sent to Meta unencoded and identically in both the authorize step and the token exchange — Instagram binds the auth code to

*that exact string, and a percent-encoded mismatch is the classic silent failure. The code also retries the exchange with the trailing slash toggled, in case the app was registered with one.*

## 1.5 Connecting YouTube

YouTube is deliberately **parallel in shape** to Instagram, with two real differences:

- **No tester allowlist.** The Google OAuth consent screen is in **Production** status (moved out of Testing on 26 Aug 2026 to kill the 7-day refresh-token expiry). Any Google account can now complete the flow. The UI still keeps a lightweight *channel review* step — you eyeball that the channel is real and suits the campaign — but there is nothing to pre-authorise in Google Cloud any more.
- **The refresh token is everything.** Google issues a ~1-hour access token plus a refresh token, and returns the refresh token *only on the first consent* unless `prompt=consent` is forced. ClipGrow forces it and **rejects any exchange that comes back without a refresh token** — an account that connects fine and then silently stops updating an hour later is far worse than one that visibly fails to connect.



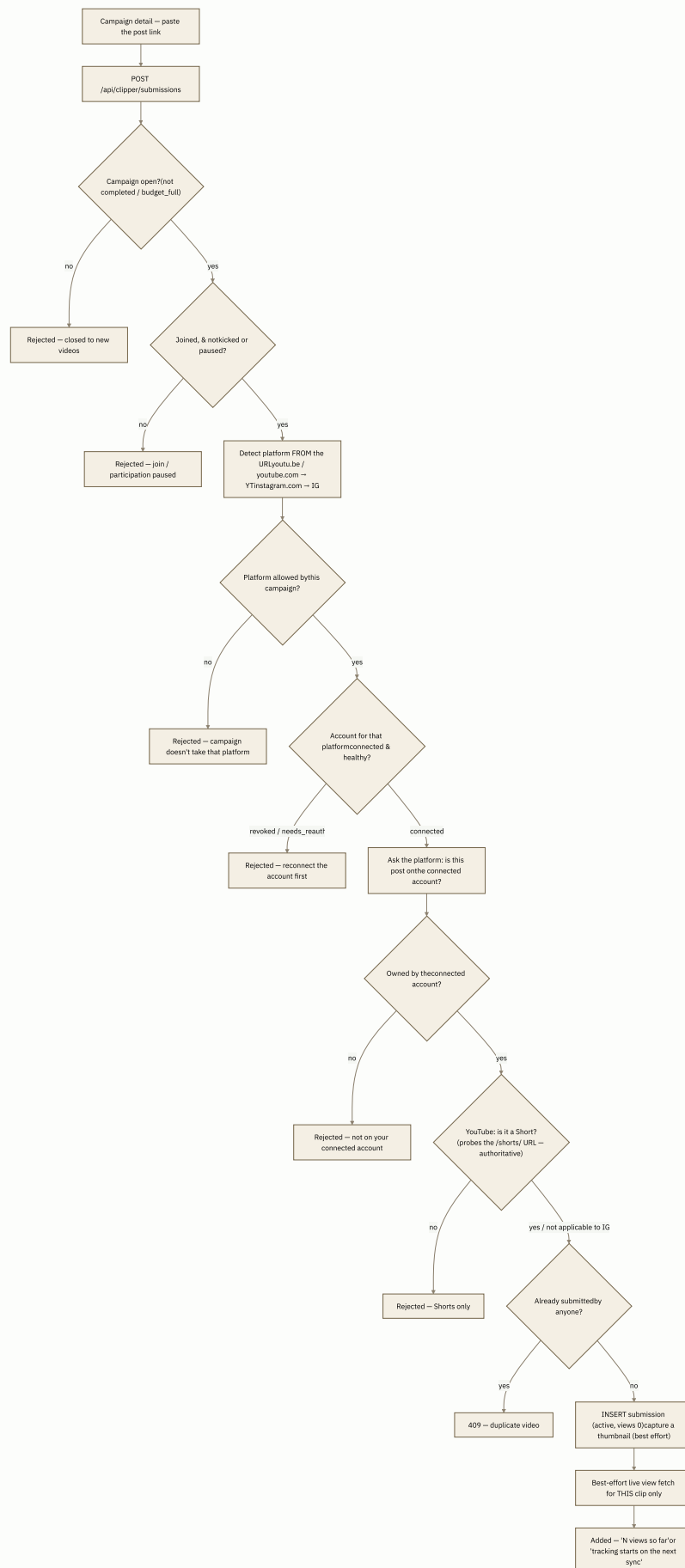
**Connect YouTube, full decision tree.** *View reads can fall back to a plain API key for public videos, so counts keep updating even while a channel's OAuth is mid-reconnect.*

## 1.6 Submitting a clip

Two ways a clip enters the system:

### Manual paste (the normal path)

The clipper posts to their account, then pastes the post link into the campaign. The platform is **detected from the URL**, never trusted from the client — a clipper cannot file a YouTube link against an Instagram account by tampering with the request.

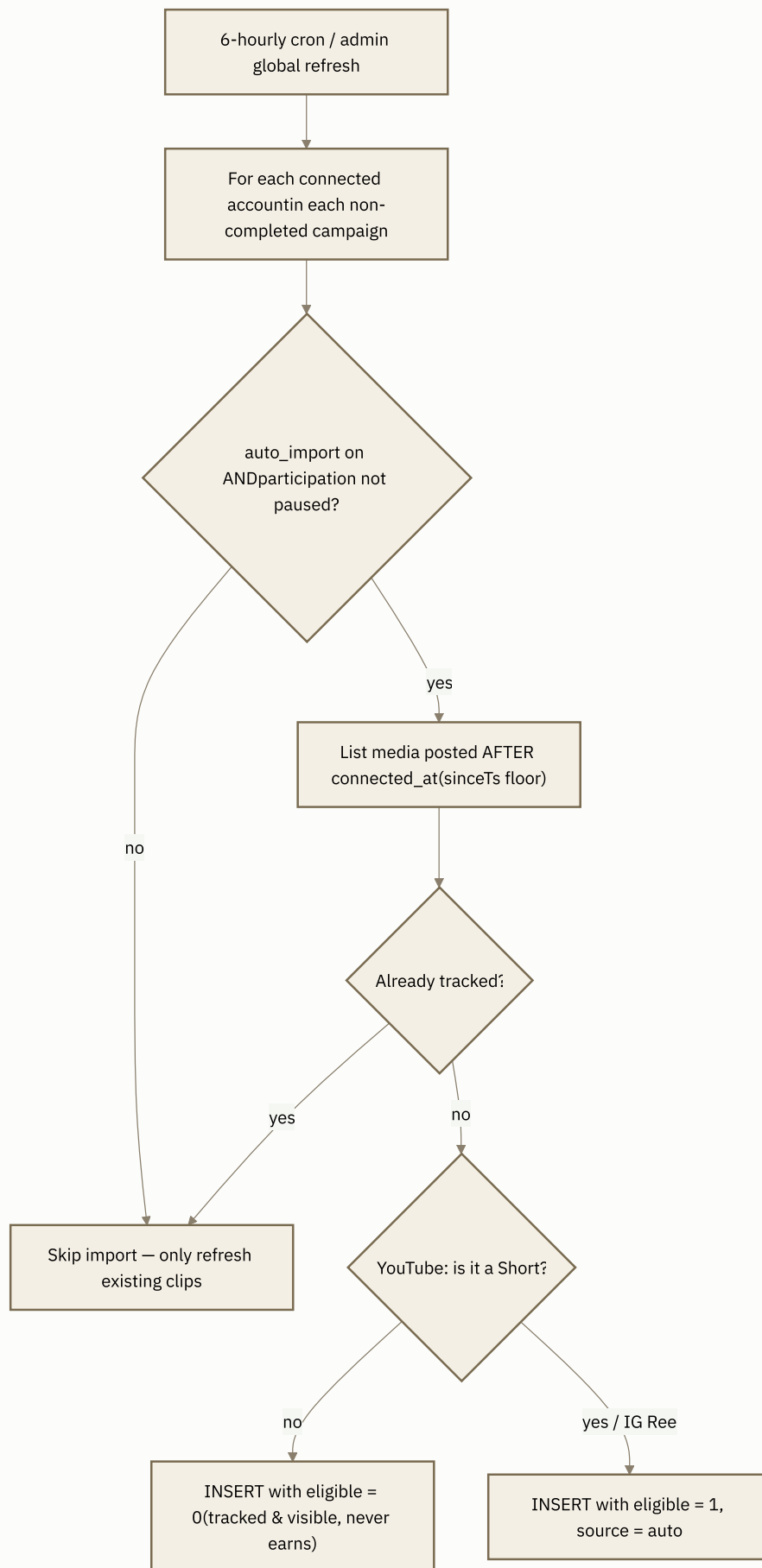


**Submit a clip.** *The ownership check and the Shorts probe are what stop a clipper claiming someone else's viral video. A brand-new post whose insights aren't ready yet still submits fine – it*

*just sits at 0 until a sync picks it up.*

### Auto-import (the cron path)

If an account has **auto-import on** (the default), the 6-hourly job also sweeps in **new** posts on its own. “New” is the load-bearing word: only posts made *after* `connected_at` are picked up (the `sinceTs` floor). This is what stops a clipper connecting an account with an old viral Reel and instantly claiming the whole budget. An account a clipper also uses personally can be set **paste-only** so their unrelated videos aren’t swept in.



**Auto-import.** *Runs as one leg of the chained refresh job below.*

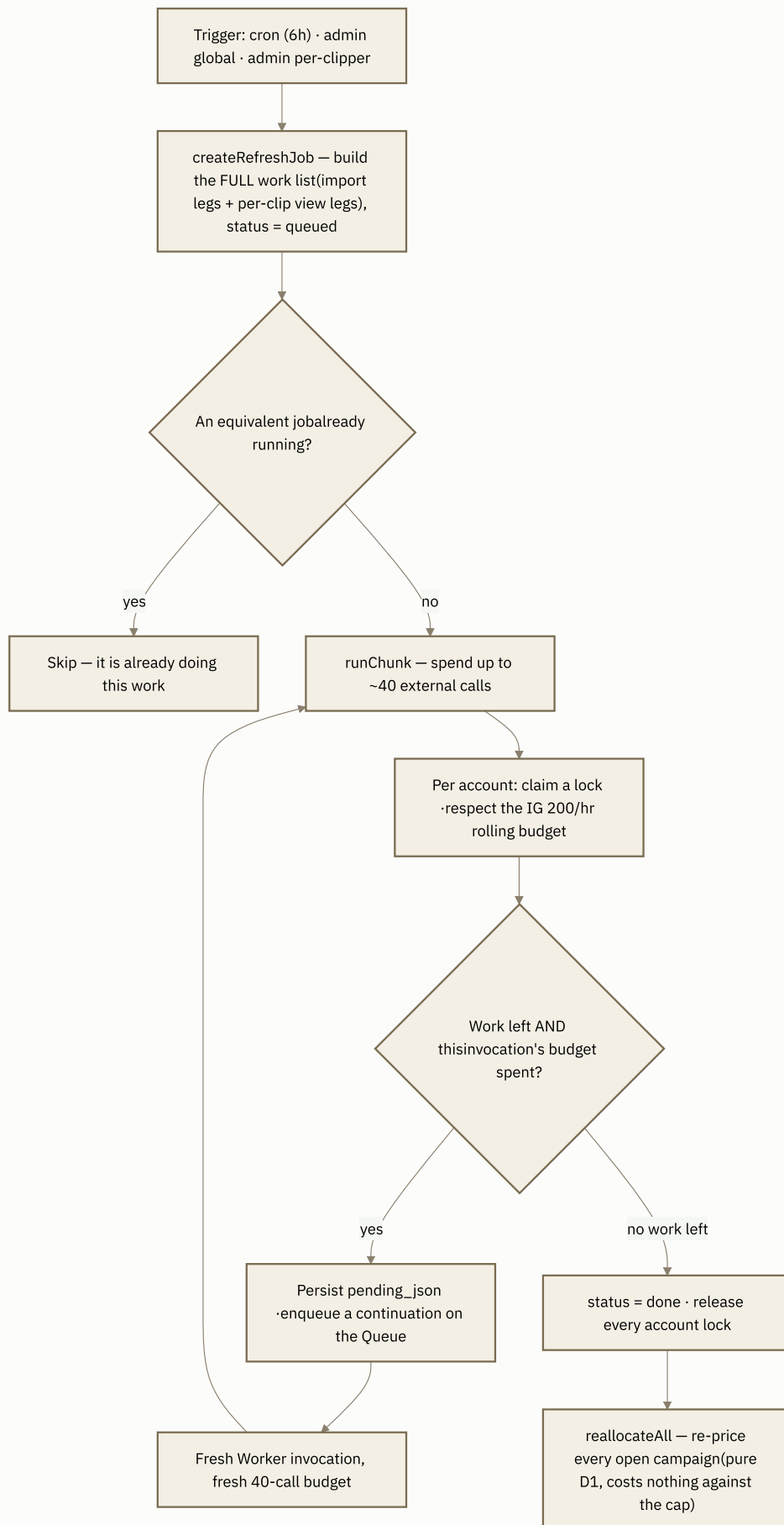


## 1.7 How views get counted

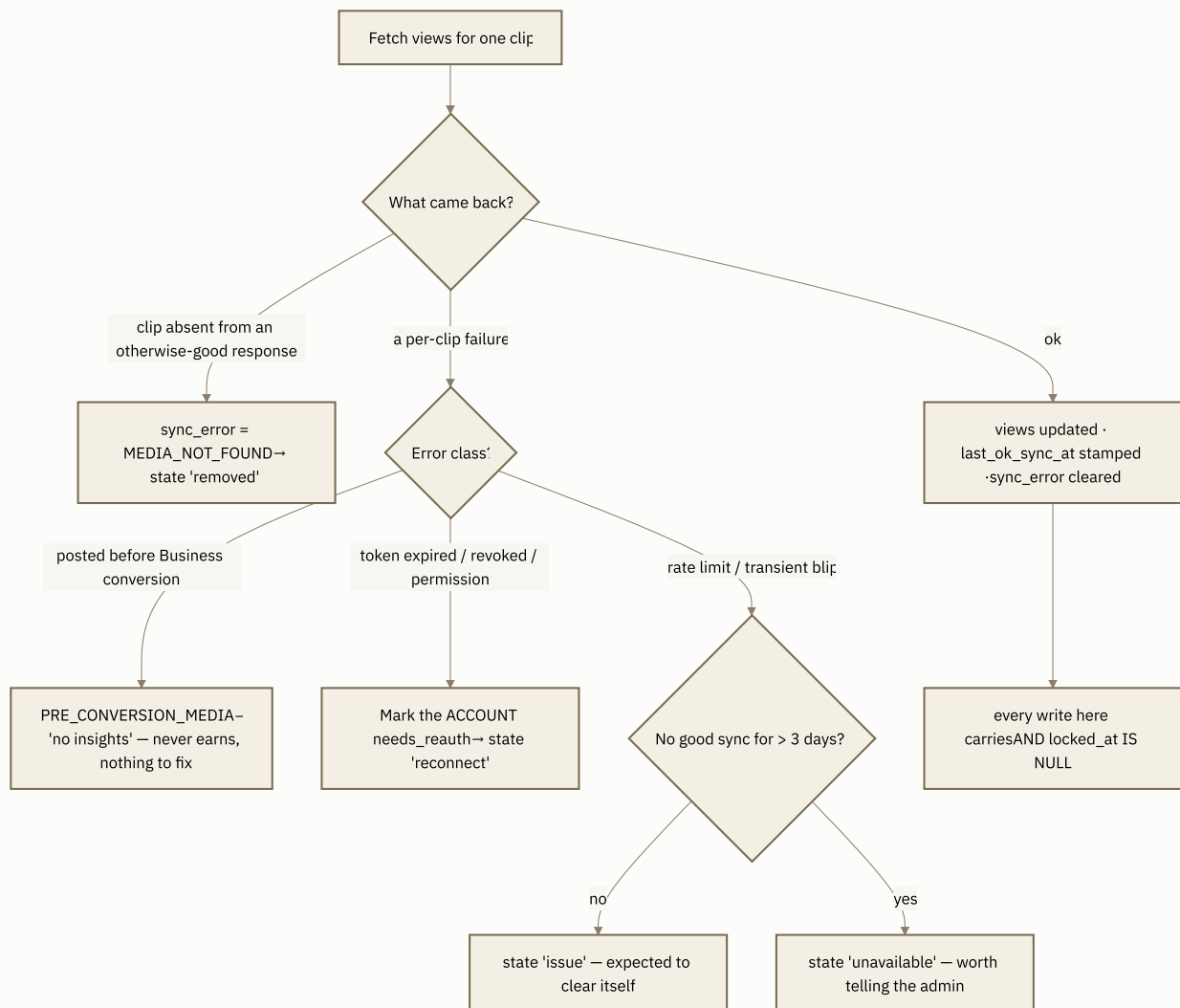
Views are refreshed by **one 6-hourly cron** and by a **per-clip “check this one now”** button. The clipper-triggered *full* refresh was removed on 27 Aug 2026 — it caused lock-contention deadlocks and the cron was already healthy. Clippers now see a countdown to the next automatic sweep.

Two ceilings constrain all of this and must not be confused:

| Ceiling                                  | What it is                                                                                                                                              | How the code respects it                                                                                                                                                                                          |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cloudflare per-invocation subrequest cap | A single Worker invocation can only make so many <code>fetch()</code> calls to the outside world (D1 doesn't count)                                     | A refresh is a <b>chained job</b> : spend ~40 calls, persist what's left, hand off to a fresh invocation via a Queue                                                                                              |
| Instagram 200 calls / hour / account     | A hard, per-account, <b>rolling</b> window (each call advances the window an hour). Batching does <i>not</i> help — Meta counts every call individually | A live ledger ( <code>ig_api_calls</code> ) of calls actually made; a per-clip 1-hour cooldown for automatic sweeps; a hard refusal if an account has more eligible clips than one hour's budget could ever cover |



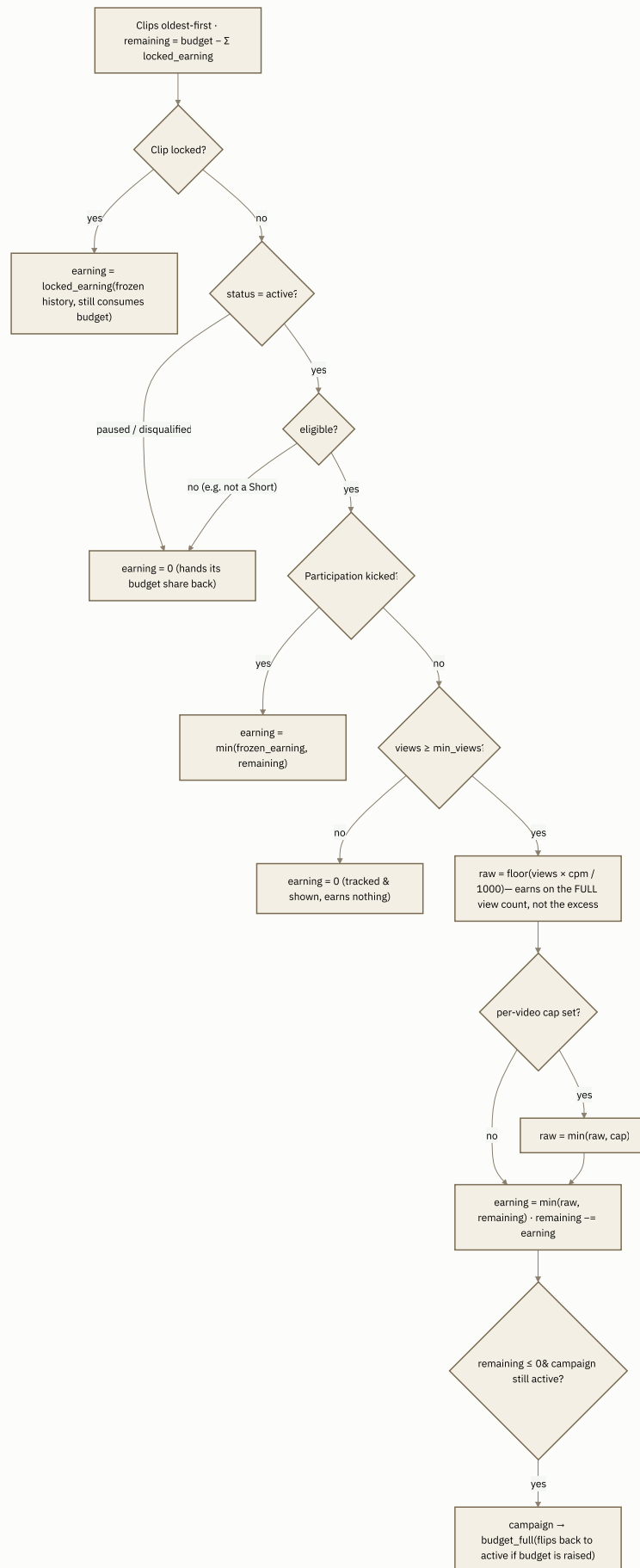
**The chained refresh job.** Because remaining work is persisted in `pending_json`, a leg that dies is simply redelivered by the Queue and re-does nothing that already succeeded.



**Per-clip sync outcome.** *The key distinction: “we haven’t heard yet” (a fresh Reel) versus “we asked and something is genuinely wrong.” One clip’s failure never touches another clip’s result — that isolation is a fix for a real incident where one bad clip froze an entire account.*

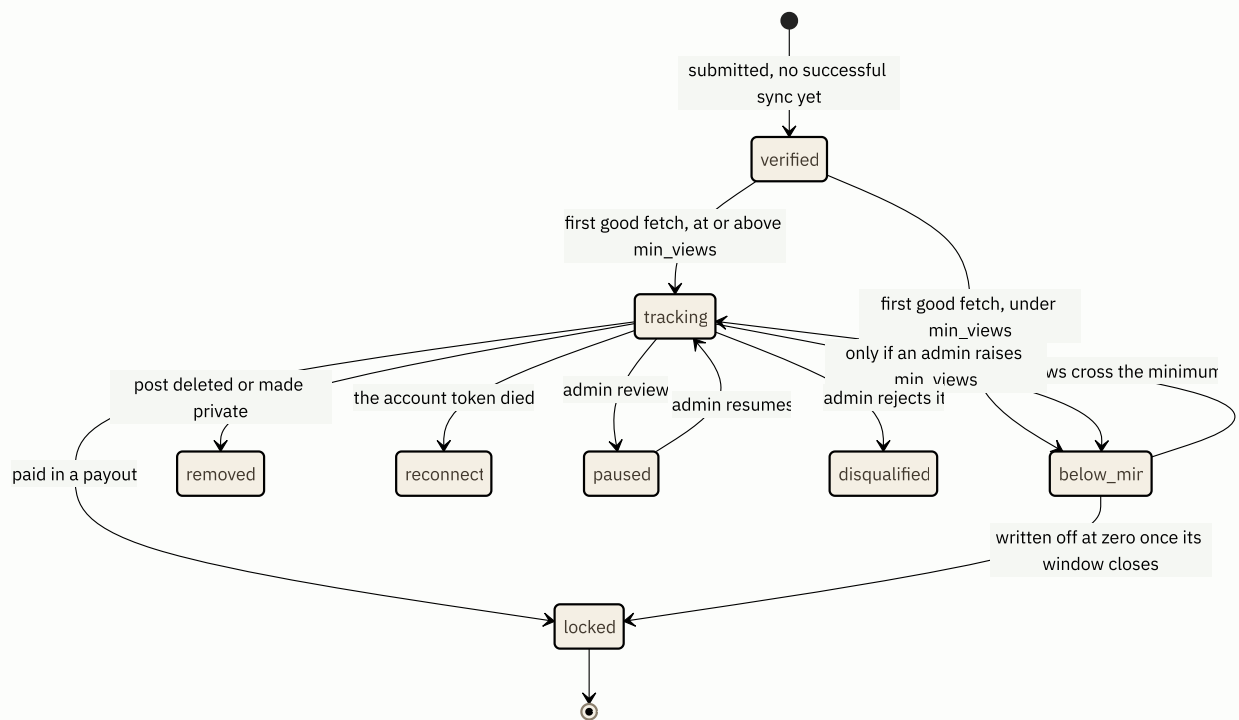
## 1.8 How earnings are calculated

After any sync, every open campaign is re-priced by `allocateCampaignEarnings`. It is a **first-come-first-served** pass over the campaign’s clips, **oldest first**, with Instagram Reels and YouTube Shorts competing for **one shared pool** on equal terms. Views only ever grow, so an earlier clip’s allocation never shrinks — later clips simply stop earning once the budget runs out.



**Per-clip allocation.** Settled money comes off the top before anything else is priced, so an older unlocked clip can never win budget a newer paid clip already spent. The multiply-before-divide in

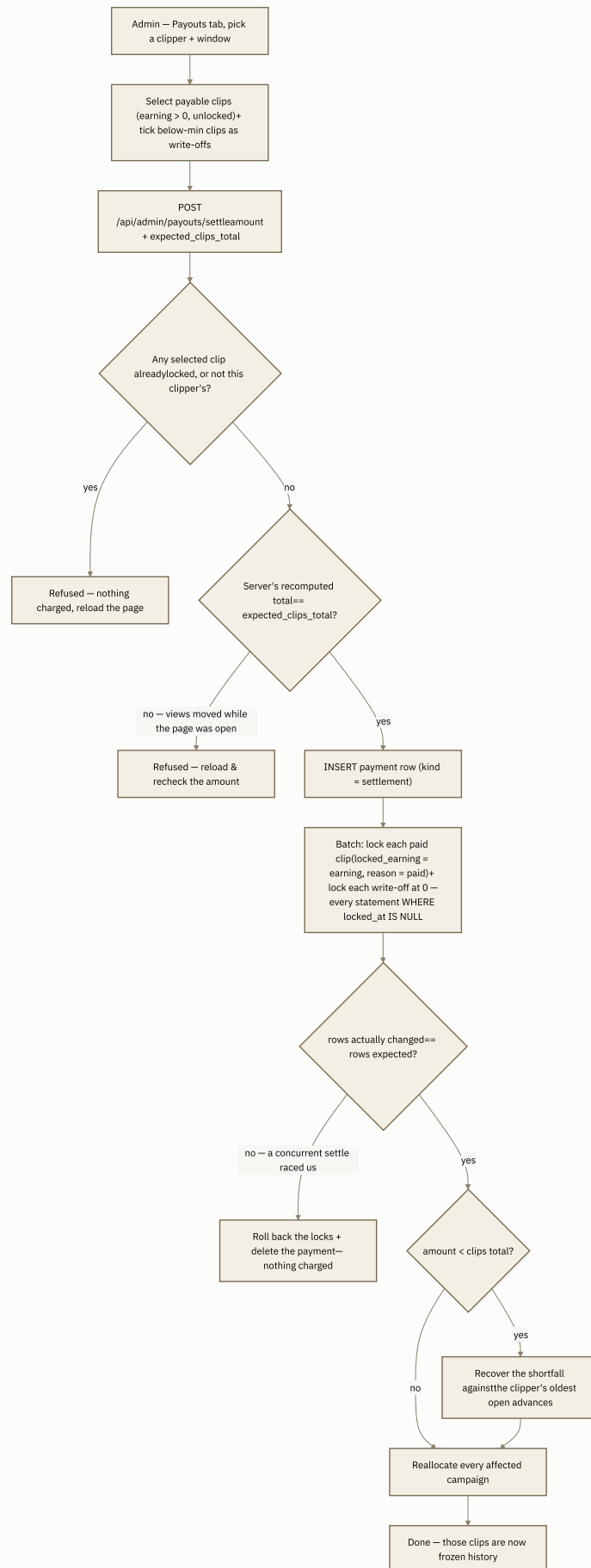
`raw` is deliberate — the other order underpays by a rupee at some view counts.



**Clip state** as derived by `clipState()` — one function, so the clipper dashboard, the admin panel and the brand portal can never describe the same clip differently.

## 1.9 Getting paid

Clippers are paid **monthly, by UPI**. You run a payout per clipper: pick a window, review each clip, settle. Settling **records the payment and locks every clip it covers in one transaction**. Locking is what stops a clip being paid twice and what makes a double-clicked *Pay* button harmless.



**Settlement.** The `expected_clips_total` check is the guard against paying an amount read off a stale page: if a clip rose from ₹120 to ₹360 between page-load and clicking Pay, the whole run is

*refused rather than transferring ₹120 and locking at ₹360.*

Three kinds of payment exist in the ledger:

| Kind                    | What it is                                    | Effect on “still owed”                                                      |
|-------------------------|-----------------------------------------------|-----------------------------------------------------------------------------|
| <code>settlement</code> | Payment for specific clips, which it locks    | Removes those clips from pending                                            |
| <code>advance</code>    | Money paid up front, before clips are settled | Deducted from what’s owed; recovered oldest-first out of future settlements |
| <code>bonus</code>      | Extra money on top of earnings                | Never deducted — it’s a gift, not an advance                                |

A clipper’s money summary ( `clipperFinancials` ): **earned** (sum of active clips) · **settled** (sum of locked earnings) · **pending** (unlocked active earnings) · **paid** (everything in the ledger) · **owed** = `max(0, pending - open advances)` .

## 1.10 When things break for a clipper

| Situation                                   | What the clipper sees                                                           | Resolution                                                                                                                   |
|---------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Token expired / revoked                     | Account goes <code>needs_reauth</code> ; a banner explains reconnecting is safe | Clipper clicks <i>Reconnect</i> on the campaign — same account, tracking resumes exactly where it left off, nothing re-added |
| Account disabled by admin                   | Read-only dashboard — every figure still visible                                | Contact admin                                                                                                                |
| Kicked from a campaign                      | Clips stop earning new money                                                    | Unpaid clips <b>freeze at their current value</b> ( <code>frozen_earning</code> ); reinstating clears the freeze             |
| Post deleted / made private                 | Clip state <code>removed</code>                                                 | Permanent — its last known views stand, it just can't update                                                                 |
| Posted before the account went Professional | Clip state <code>no insights</code>                                             | Permanent Instagram limitation — it can never earn, reconnecting does nothing                                                |
| Admin pauses / disqualifies a clip          | <code>paused</code> / <code>disqualified</code>                                 | Earns 0 while paused; disqualified is permanent. Both hand their budget share back                                           |

---

## PART 2

# The Client (brand)

A brand's account is a **read-only observer login**, scoped to the campaigns you explicitly grant it. The whole portal is GET-only by design.

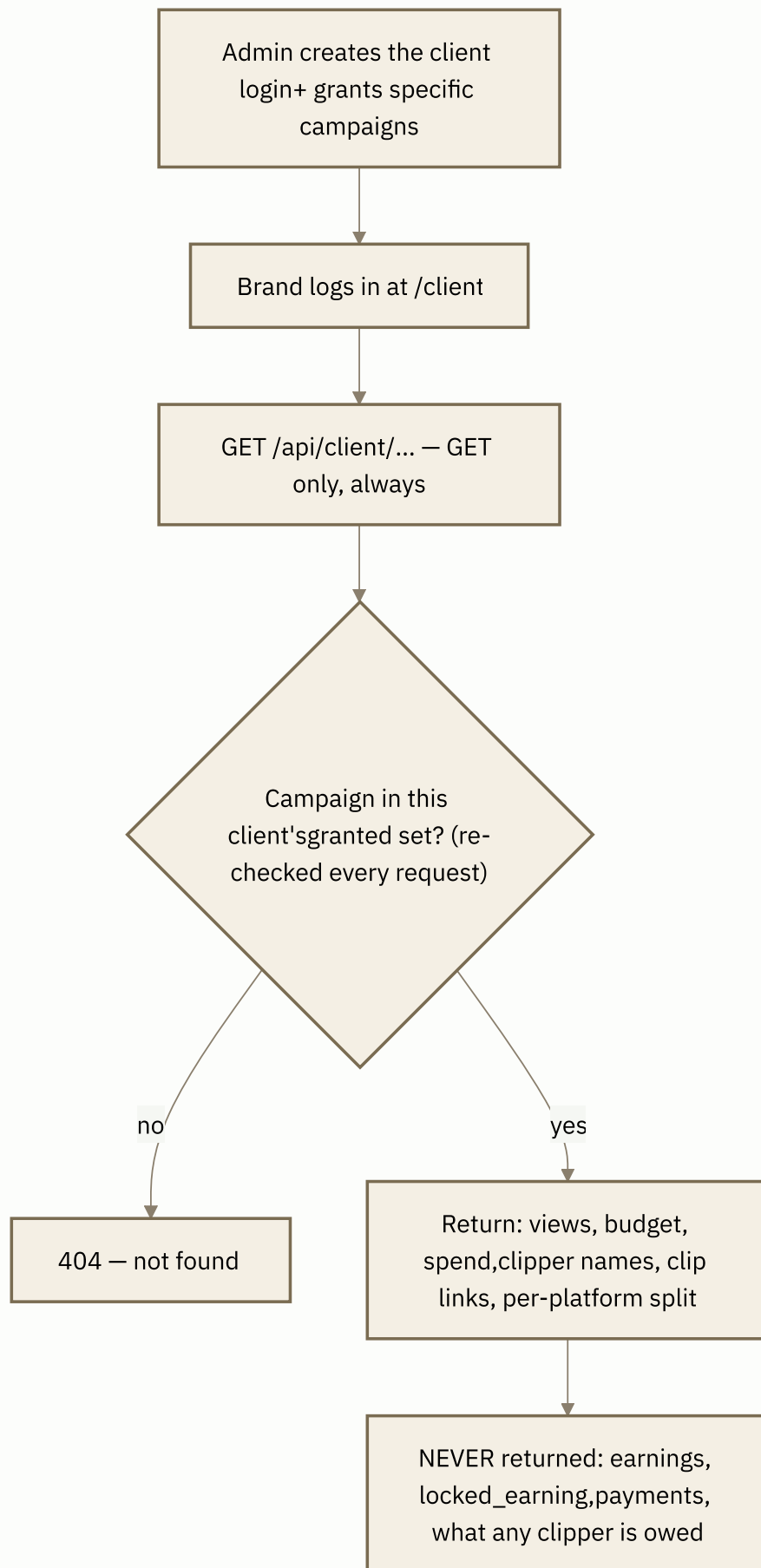
## 2.1 Onboarding & the read-only portal

You create the client in the admin panel (username, password, company name) and tick which campaigns they may see. Two boundaries are enforced on **every single request**, not once at login:

- **Scope** — the campaign must currently be in that client's `client_campaigns` grant. Untick it in the admin panel and their open session loses access immediately.



- **Secrecy** — no clipper payment state ever crosses this boundary. The client sees budget and spend (they're funding it), view counts, clipper display names and clip links. They **never** see `earning` , `locked_earning` , or anything from the payments ledger.



**The client boundary.** *A GET-only portal with per-request scope and a hard secrecy line.*

## The Admin (you)

Login is a single password ( `ADMIN_PASSWORD` ) → role `admin` cookie. The panel has six tabs:

**Campaigns, Clippers, Payouts, Payments, Clients, Authentication.**

### 3.1 Creating a campaign

1. Optionally upload the brand's brief as a **Word** `.docx` — `parseBlueprintDocx` pulls out objective, guidelines, footage links, CTA, tags and the per-video cap into a draft.
2. Set the money: **CPM** (> 0), **budget** (> 0), **min views** (defaults to 1,000), **allowed platforms** (Instagram and/or YouTube).
3. Save. A permanent URL slug is generated from the name + id, so a shared `/campaigns/:slug` link never breaks even if the campaign is renamed.

Editing CPM, budget, min-views or the per-video cap **immediately re-prices every clip in that campaign**. Zeroing the budget to freeze a campaign is a real, supported move (a deliberate `0` is distinguished from “field not sent”).

### 3.2 The Authentication queue

This tab is the manual gate in front of OAuth. Requests are sorted `requested` first. For each:

- **Instagram** — add the handle as a Tester in the Meta App Dashboard by hand, then mark the request `confirmed`. The clipper's *Connect* button appears.
- **YouTube** — open the channel, check it's real and on-brief, mark `confirmed`. (No Google Cloud step any more.)
- The row shows whether that exact account was **already approved on another campaign** (tester status is app-level — if so, just confirm) and whether the clipper has **since connected** (so a stale row reads as resolved).

### 3.3 Moderation levers

| Lever                       | Effect                                                                                                                                                   |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clip → <b>paused</b>        | Temporarily not monetised (under review). Earns 0, budget share returned. Reversible.                                                                    |
| Clip → <b>disqualified</b>  | Permanently rejected. Earns 0.                                                                                                                           |
| Participant → <b>paused</b> | No new clips (manual or auto-import). Existing clips keep tracking <i>and</i> earning.                                                                   |
| Participant → <b>kicked</b> | Unpaid clips freeze at current value via <code>frozen_earning</code> . Money still owed, still consumes budget. Reinstating clears the freeze.           |
| Account → paste-only        | Auto-import off for that account — for a clipper posting campaign work from a main account they also use personally.                                     |
| Disconnect an account       | Unlinks it, deletes its <i>pending</i> clips (budget returned), <b>keeps every settled clip</b> and the account row with them so history still resolves. |

Guard rails: you cannot delete a clipper or campaign with any submissions (disable / mark-over instead); you cannot edit or delete a clip attached to a real payment (reverse the payment first).

### 3.4 Running a payout

- **Payouts tab** → pick a clipper → pick a window (30 days / all time / one campaign). You see each clip with its posted & synced dates, whether it cleared the minimum, whether the cap bit, and what's already settled.
- **Write-offs** — clips still under the minimum whose window has closed get closed at zero (`lock_reason = 'below_min'`). Money-neutral (they earned 0 anyway); reversible per-clip. There's also a one-click sweep for old below-min clips across everyone.
- **Reverse a payment** — deletes the payment and unlocks every clip it locked; they return to the pending pool and re-price. This is the escape hatch for a mistyped amount. Deleting a payment routes through the same reversal so clips are never stranded locked.
- **Advance / bonus** — recorded on the Payments tab; these move money *without* settling any clips.

### 3.5 Sync & diagnostics

- **Global refresh** — same chained job as the cron, but ignores the per-clip cooldown (“show me the truth right now”). Same lock as a per-clipper refresh, so the two can’t collide.
- **Per-clipper refresh** — every account in every campaign that clipper is active in.
- **Retry** — a job stuck `running` with no progress for 10 minutes is `stalled` ; retry re-queues it from `pending_json` .
- **Health banner** — names the specific broken account: accounts needing reauth, accounts whose auto-import is failing (existing clips still sync — only *new* uploads silently stop arriving), and clips stuck with an error for 12+ hours.
- **Diagnostics** — an Instagram-metrics probe (every metric Meta will answer, side by side with the stored value) and a media audit (every post on an account vs. what ClipGrow tracks, to catch an untracked post or a deleted-and-reposted clip). Every diagnostic call is counted against the account’s budget like any other.
- **CSV export** — clips or payments, generated live from D1, so a download is always a true current snapshot even if the site is down.

---

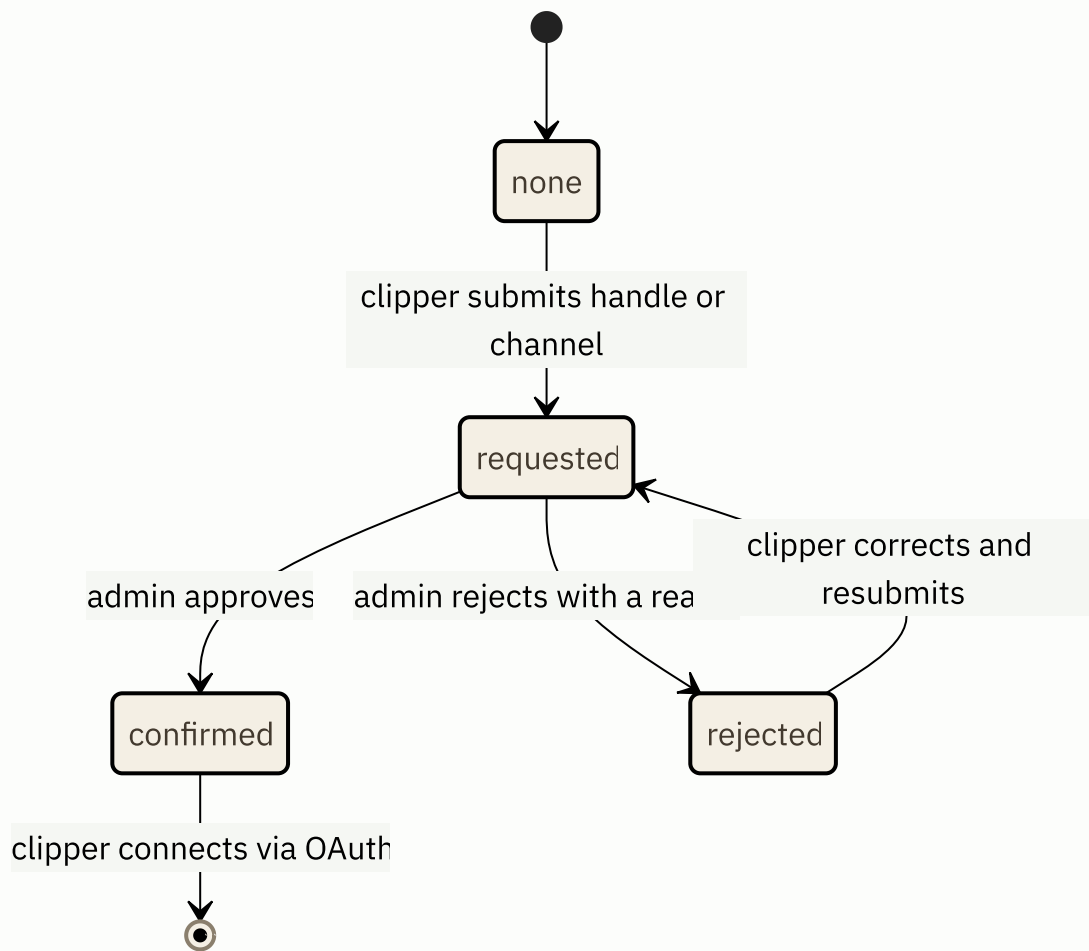
## PART 4

# Every state machine, in one place

Seven things in ClipGrow have a lifecycle. Three appear above in context; all seven are collected here for reference.

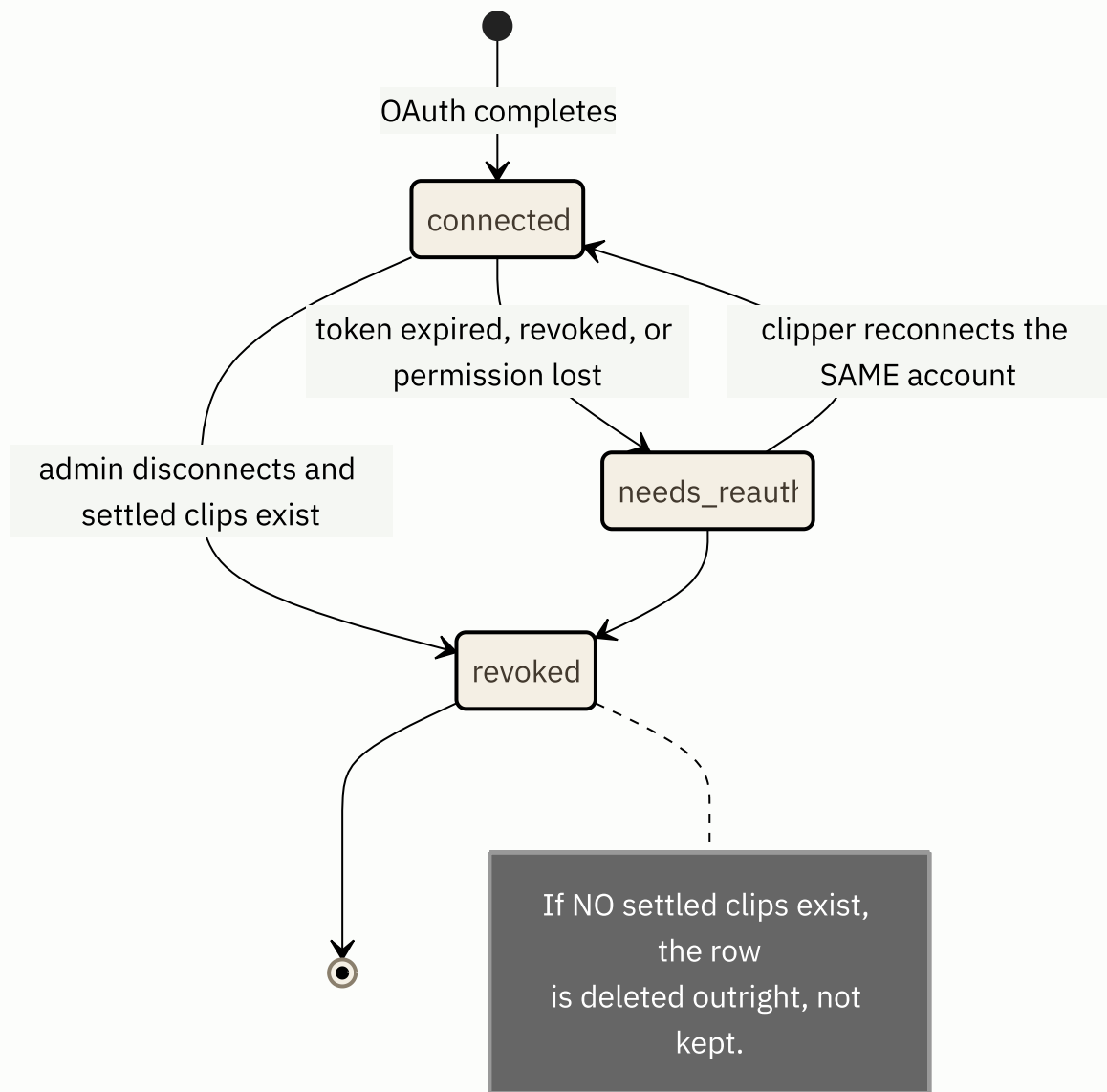
### 4.1 The seven state machines

Access request `tester_requests.status`



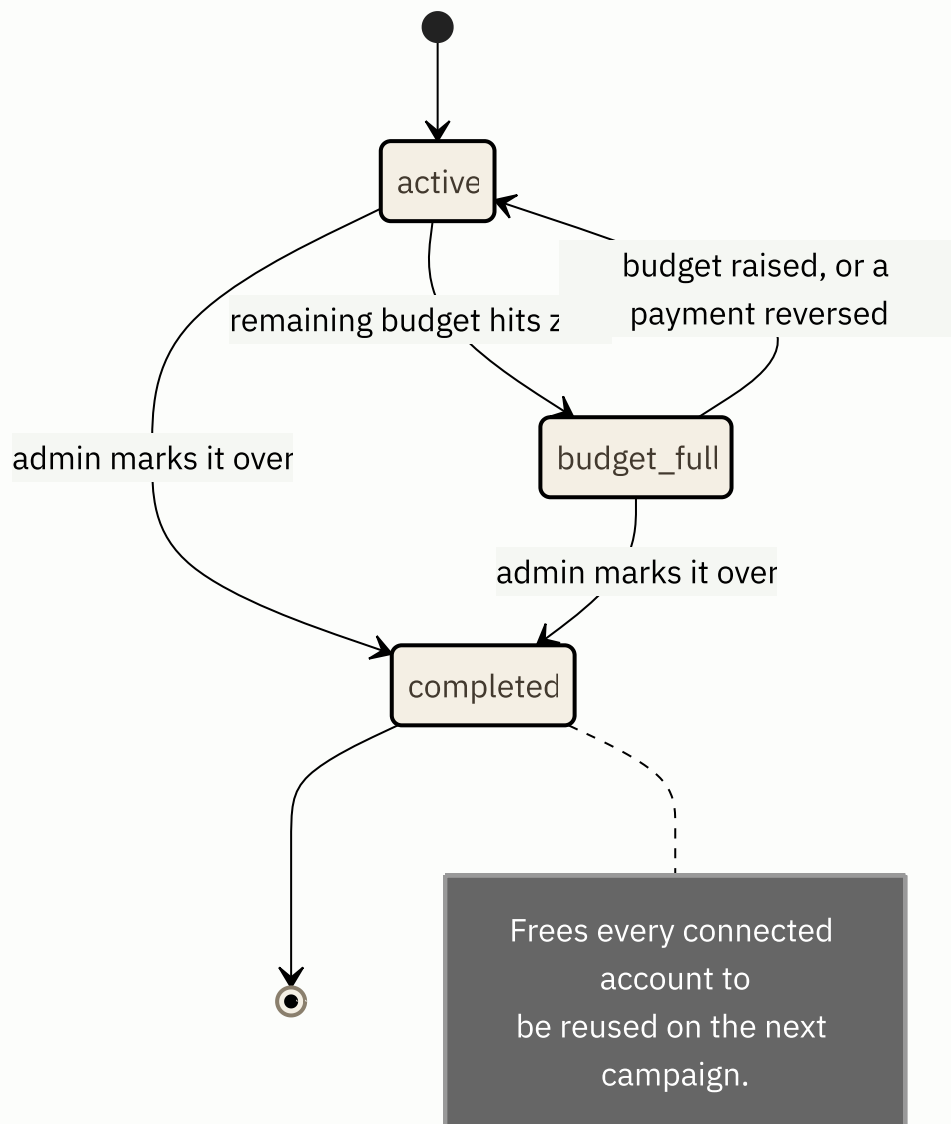
*Instagram enforces this fully. YouTube uses it as a lightweight channel review only.*

**Social account** `social_accounts.status`



*Instagram tokens last 60 days and are renewed by a scheduled job. Google access tokens last an hour and are renewed inline on every call.*

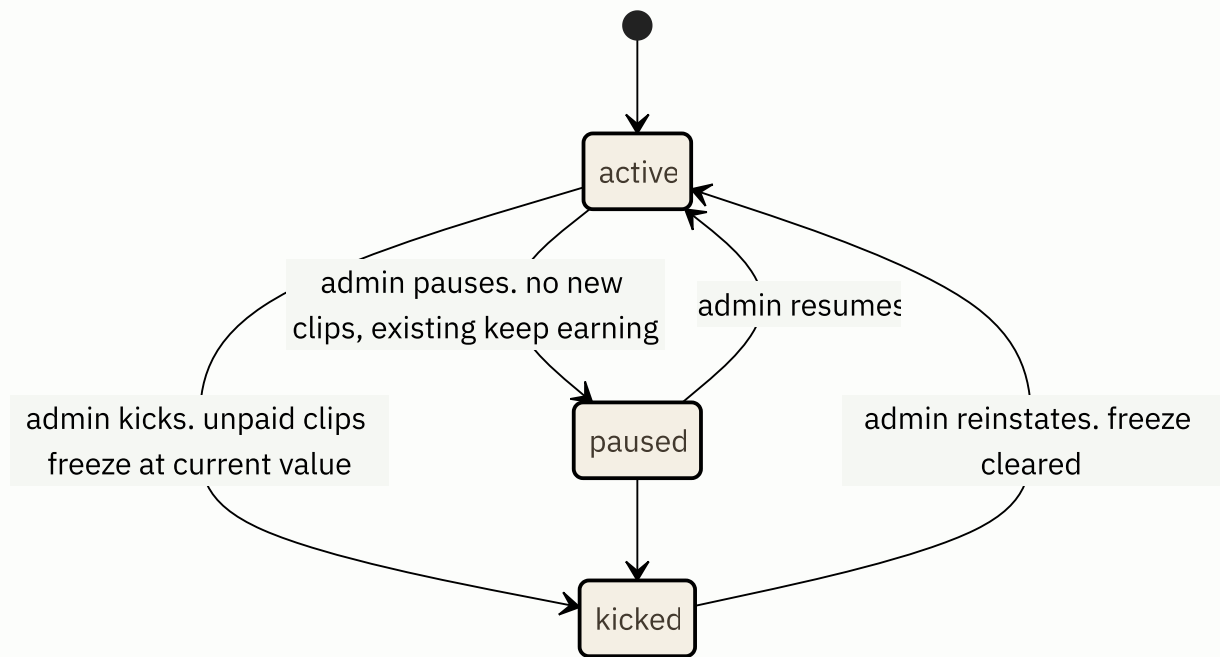
**Campaign** `campaigns.status`



The `active ↔ budget_full` flip is automatic, done by the allocator on every re-price.

Participation `participations.status`



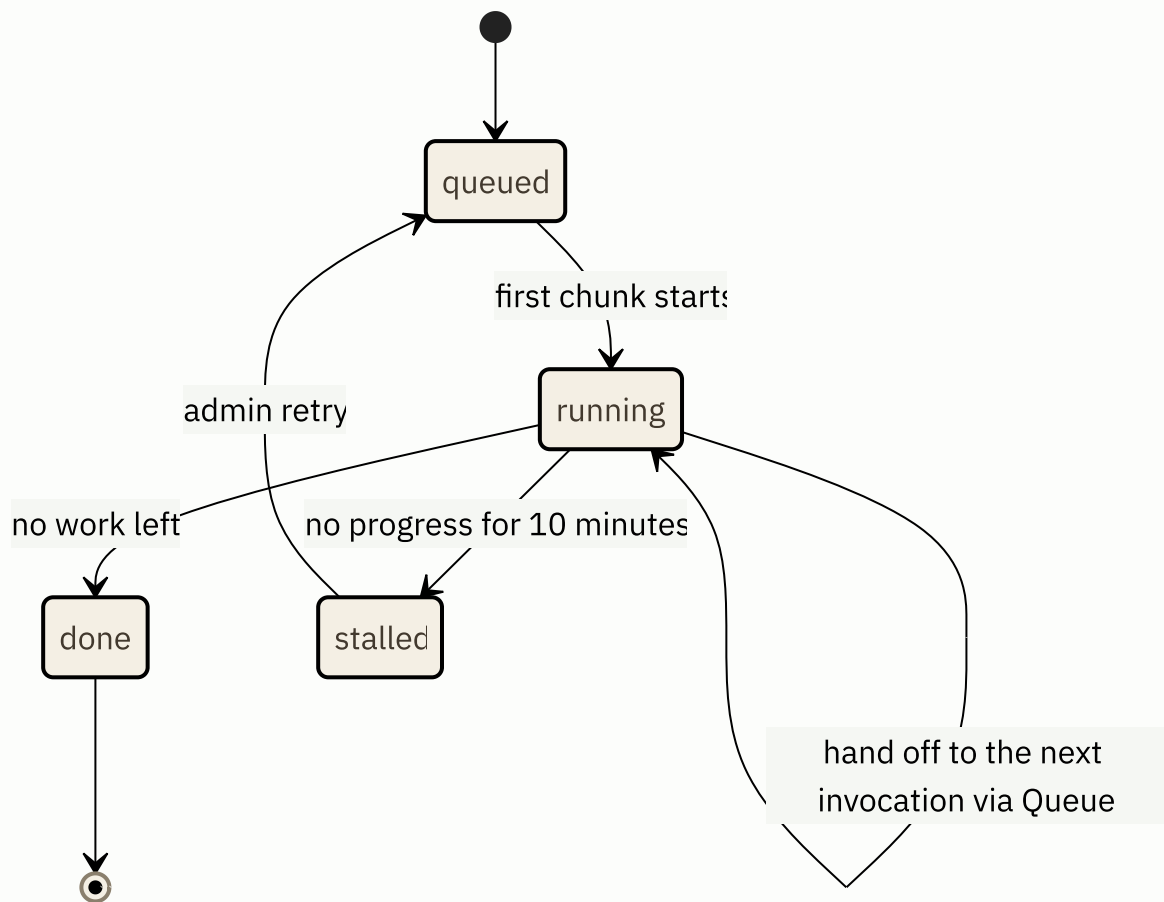


### Clip / submission `clipState()`

Derived, not stored — see §1.8. Healthy path: `verified` → `below_min` / `tracking` → `locked`.

Error states: `removed`, `no_insights`, `reconnect`, `issue`, `unavailable`, `paused`, `disqualified`, `ineligible`.

### Refresh job `refresh_jobs.status`



## Payment `payments.kind`

Not a lifecycle so much as a type tag set at creation: `settlement` (locks clips), `advance` (recovered from future settlements), `bonus` (never recovered). A payment is either edited, or reversed (which unlocks its clips). There is no “pending payment” state — the row is written only after the money has actually moved.

# Invariants — the things that must always hold

## 5.1 The safety rules

| #  | Invariant                                                                                                                                      | Why it exists                                                                                             |
|----|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1  | Every write that can move an <code>earning</code> carries <code>AND locked_at IS NULL</code> , and the caller checks <code>meta.changes</code> | A locked clip is settled financial history; a mid-sync write would desync the CSV audit trail             |
| 2  | Earnings are computed server-side from the clips, never trusted from the client. Settlement also checks <code>expected_clips_total</code>      | Views move between page-load and clicking Pay; without this you'd pay the old number and lock the new one |
| 3  | A clip's platform is detected from its URL                                                                                                     | Stops a clipper filing a YouTube link against an Instagram account                                        |
| 4  | One social account drives one <i>live</i> campaign at a time, checked across all clippers                                                      | Two logins can't quietly point at the same account; a completed campaign frees it                         |
| 5  | Auto-import only picks up posts made after <code>connected_at</code>                                                                           | Stops connecting an old viral post and instantly claiming the budget                                      |
| 6  | OAuth <code>state</code> is HMAC-signed, 10-minute TTL, and its <code>sub</code> must equal the current session                                | A connection link can't be replayed or used from a different login                                        |
| 7  | Instagram <code>redirect_uri</code> is byte-identical (unencoded) in authorize and token exchange                                              | Instagram binds the auth code to that exact string; a mismatch fails the exchange silently                |
| 8  | A YouTube connection with no refresh token is rejected outright                                                                                | It would connect fine and then stop updating an hour later — worse than a visible failure                 |
| 9  | Locking is idempotent — a clip can't be locked twice                                                                                           | A double-clicked <i>Pay</i> is harmless; a raced settle rolls itself back                                 |
| 10 | One <code>clipState()</code> function feeds clipper, admin and client views                                                                    | They can never describe the same clip differently                                                         |

| #  | Invariant                                                                               | Why it exists                                                                             |
|----|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 11 | Instagram calls are metered against a live 200/hr rolling ledger; diagnostics count too | A diagnostic that spent untracked quota would corrupt the ledger it's used to investigate |
| 12 | One clip's fetch failure never affects another clip on the same account                 | A single bad clip once froze an entire account's view counts                              |

## PART 6

# Where the model is currently fragile

As of **late August 2026** — verify against `.claude/notes/whats-next.md` and the git log before acting.

## 6.1 Open items

### BLOCKING INSTAGRAM AT SCALE

The Meta app is still in **Development Mode**, so the Tester allowlist in §1.4 is real — every Instagram clipper needs you to add them by hand. Meta Business Verification (the way out) is blocked on two things on your side: **registering ClipGrow as a legal entity** (Udyam/MSME is the fast path) and a working `support@clipgrow.in` mailbox (Cloudflare Email Routing, ~10 min).

### INSTAGRAM TOKEN EXCHANGE

A prior session recorded the Instagram OAuth **token exchange failing**, isolated to the `IG_CLIENT_SECRET` value in the Worker settings. If Connect Instagram is still failing at the “exchange code → token” step (§1.4, node `M`), re-check that secret first. See memory `project-clipgrow-instagram-oauth-blocker`.

### YOUTUBE ONE-TIME RECONNECT

Moving the Google app from Testing to Production on 26 Aug forced every existing YouTube connection to re-auth once. **ranjith** and **sam** had not done theirs as of 27 Aug — worth a nudge. Check with the `wrangler d1 execute` query in `whats-next.md`.

## STRUCTURAL

Free-tier Cloudflare limits are why the refresh engine is as complex as it is (§1.7). The 6-hourly cron is the only thing refreshing views now — if it silently stops, view counts freeze and nobody is alerted except through the admin health banner. It has been healthy (6/6 recent runs clean) but it is a single point of failure.

---

Compiled from the `feature/clipper-platform` source on 28 Aug 2026. The diagrams show behaviour as coded, not as aspirationally designed — where a flowchart and a comment in the source disagree, trust the source and tell me. Everything here is derived from `src/`, the `.claude/notes/` files, and saved memory.